The SAFER Protocol

Security and Fire Emergency Reports

Next-Generation Reporting for Modern Alarm Systems

Version 1.0

February 2026

© 2024 Dan Horon
Licensed under the Apache License, Version 2.0
U.S. Copyright Registration No. TXu 2-426-289
Effective March 7, 2024

Contents

Introduction 1
Context and Rationale 2
About the Author 3
Executive Summary: Highlights of the SAFER protocol 4
Advantages of the SAFER Protocol 5
The SAFER Protocol 6
SAFER Protocol Structure 9
Sample Formatted Template 13
End-User Interaction 15
Implementation Guidelines 16
Software Development Integration 18
Compliance and Testing Requirements 21
Future Prospects and Upgrades 23
Reference Materials and Resources 28
Contact Information 29

Introduction

The **S**ecurity **A**nd **F**ire **E**mergency **R**eports (**SAFER**) protocol defines a structured, JSON-based message format for reporting life-safety and security events across modern IP networks.

Originally authored in 2024 and published for public use under the Apache License 2.0, SAFER is intended to support interoperability, extensibility, and precise event representation in fire alarm and security systems.

SAFER standardizes the event payload only. Transport mechanisms, regulatory compliance, listing requirements, and system performance characteristics remain the responsibility of implementers.

The intent of SAFER is to provide:

- A human- and machine-readable event structure
- Precise identification of customer, panel, and device
- Extensibility without breaking backward compatibility
- A migration path from legacy signaling formats

SAFER is designed to support long-term evolution and collaborative improvement within the life-safety industry.

Context and Rationale

The introduction of the SAFER protocol comes during a pivotal moment in the fire alarm industry—a field born in the mid-1800s when Morse Code was used to communicate alarms automatically.

As technology evolved, alarm signals were sent via dial-up telephone lines. An ideal solution when first introduced, over time, telephone technology is no match for today's technological superhighway.

Until recently, the fire alarm industry has relied almost exclusively on dedicated wiring systems to ensure reliable and secure communication between buildings, and between devices. Hard-wired technology was the industry standard in the 1990s. But, compared to Ethernet, a twisted pair of wires is inherently limited. Moreover, traditional reporting standards like Contact ID are slow and limit the amount of information that can be provided to emergency responders.

Today, the world has moved from traditional, isolated systems toward more integrated, adaptable, and resilient network infrastructures—which means we can also continue to evolve fire alarm technology.

The SAFER protocol is specifically designed to leverage the strengths of Ethernet-based communication. This transition can have a strategy that bridges the gap between established fire alarm systems and modern IT-driven building management systems.

The SAFER protocol recognizes that the technological advances of the past are a foundation for the future, and that as an industry, we continually aspire to elevate industry standards.

The SAFER protocol is a set of guidelines to support the industry's dedication to evolution, safety, and resilience.

The SAFER protocol provides a structured framework for modernizing event reporting while preserving compatibility with established supervisory practices.

About the Author

Dan Horon is a long-standing member of the NFPA Technical Committee on Protected Premises and has participated in task group work related to circuits, pathways, and network communications within fire alarm systems.

His involvement in standards development has included work associated with the evolution of performance-based circuit descriptions and the introduction of network concepts such as Class N in NFPA 72.

The SAFER protocol reflects ongoing efforts to clarify and modernize digital event reporting structures in alignment with contemporary IP-based communication systems, and is intended for collaborative development with industry participation.

Future revisions may incorporate feedback, contributions, and implementation experience from manufacturers, integrators, end-users, and standards participants.

Executive Summary: Highlights of the SAFER protocol

The SAFER protocol defines a structured, JSON-based format for transmitting fire alarm and security events over modern IP networks.

SAFER is designed to modernize event reporting by replacing legacy signaling formats with a flexible, extensible, and machine-readable structure. It standardizes the event payload while remaining transport-agnostic and compatible with existing supervisory and regulatory frameworks.

Key characteristics of the SAFER protocol include:

- **Structured Event Representation** – Human- and machine-readable JSON format.
- **Precise Identification** – Clear separation of customer, panel, device, and condition.
- **Extensibility** – Optional fields allow expansion without breaking backward compatibility.
- **Interoperability** – Designed for implementation across manufacturers and monitoring platforms.
- **Migration Path** – Enables transition from legacy formats such as Contact ID to modern network-native reporting.

SAFER does not define pathway supervision, transport reliability, listing requirements, or system performance criteria. Those remain governed by applicable codes, standards, and product certifications.

The protocol provides a consistent event structure intended to support long-term interoperability, clarity, and evolution within life-safety and security systems.

Advantages of the SAFER Protocol

The SAFER protocol provides the following structural advantages:

- **Expanded Data Capacity** – Supports structured, extensible event payloads beyond legacy fixed-length signaling formats.
- **Structured Identification** – Separates customer, panel, device, and condition into discrete fields for clarity and precision.
- **Transport Agnostic** – Designed to operate over IP-based networks without dependence on a specific transport mechanism.
- **Backward Reference Capability** – Allows legacy event formats (e.g., Contact ID) to be embedded as optional reference fields.
- **Extensible Schema** – Optional fields enable implementation-specific extensions without breaking baseline compatibility.
- **Human- and Machine-Readable Format** – JSON structure facilitates parsing, logging, diagnostics, and long-term archival use.
- **Interoperability Potential** – Provides a common event structure that may be adopted across manufacturers and monitoring systems.

The SAFER protocol does not replace pathway supervision requirements, equipment listings, or regulatory compliance obligations. It standardizes event structure only.

The SAFER Protocol

The SAFER protocol defines a structured JSON-based event format designed for use in fire alarm and security reporting systems. The protocol specifies required and optional fields and establishes version-controlled message identification.

Leveraging JSON

The SAFER protocol leverages the adaptability and structure of the JSON programming language to ensure detailed, precise communication of alarm states and device specifics. The JSON format is so basic and versatile that it will likely be long-lived.

Technical Background: A Brief Explanation of JSON Format

JSON (JavaScript Object Notation) is a lightweight data interchange format that's easy for people to read and write, and simple for machines to parse and generate. It's based on a subset of JavaScript language but is language-independent, making it an ideal data format for a wide range of applications.

JSON structures data in name/value pairs and ordered lists, providing a clear, hierarchical representation that's particularly well-suited for complex data interactions in modern web applications.

SAFER Alarm Event Template

The SAFER alarm event template offers a comprehensive and detailed approach to reporting alarm events. It includes essential information: customer ID, panel ID, and device ID, and expands on traditional reporting by incorporating named device types, equipment locations, and statuses. The template also allows for optional, more granular data such as GPS coordinates, system categories, and a link to additional resources. This structure ensures that each alarm event is reported with maximum clarity and context, enhancing the responsiveness and precision of emergency services and system monitoring.

The SAFER protocol enhances the clarity and accuracy of alarm data and provides a flexible framework compatible with various devices, facilitating seamless integration and future expansion in the evolving landscape of fire safety technology.

Here is an example alarm message using touch-tone coded information defined by Contact ID:

```
1234 18 1101 01 015 8 (Spaces added for clarity here)
```

A person (or software) able to read Contact ID could read this message and know that customer 1234, has an active personal emergency, sent by a sensor or switch identified as 01-015.

Here is what the same alarm could look like in SAFER:


```
{
  "CUSTOMER": "1234",
  "PANEL_ID": "Node 01",
  "DEVICE_ID": "01-015",
  "DEVICE_TYPE": "Pendant Transmitter",
  "DEVICE_LOCATION": "Indep.Lvng Unt#7",
  "STATUS": 1,
  "SAFER_ID": {
    "version": "0.0.0",
    "unique_id": "123e4567-e89b-12d3-a456-426614174000"
  },
  "OPTIONAL": {
    "PANEL_GPS": {
      "latitude": "34.0522",
      "longitude": "-118.2437",
      "altitude": "30m"
    },
    "LOCATION": "First Floor",
    "CONDITION": "Pendant Pulled",
    "OWNER_LOCATION": "Independent Living Unit 7",
    "DATE": "2024-01-01",
    "TIME": "00:00",
    "SYSTEM_CATEGORY": "Personal Emergency",
    "ZONE": {
      "name": "Zone 6F",
      "description": "",
      "url": "https://www.cvs.com/"
    },
    "GROUP": {
      "name": "Senior Living Community",
      "description": "",
      "url": "https://www.cvs.com/"
    },
    "GPS_COORDINATES": {
      "latitude": "34.0522",
      "longitude": "-118.2437",
      "altitude": "30m"
    },
    "CONTACT_ID": {
       "version": "SIA DC-05-1999.09",
       "code": "1234181101010158"
    }
  }
}
```


JSON vs. Contact ID

- Previous standards like Contact ID faced limitations due to their fixed-format nature and limited data capacity. SAFER is expandable with new optional fields, and version control allows for revisions.

- Contact ID can't represent the detailed device-specific information required by modern "smart" alarm systems. New systems have multiple Panels, each with loop numbers and individual point numbers. The panel or a printer on site may show a point as N54L32D001. Not having to reduce the point's identity to a few numbers is invaluable. SAFER uses a text field, making "N54L32D001" or any other identifier very easy to transmit intact. Contact ID's rigid structure made it difficult to incorporate new types of data or adapt to the nuanced needs as systems advanced, leading to challenges in precise identification, location tracking, and comprehensive status reporting of individual devices in complex environments. SAFER offers a structure to organize data, and has no such limitations.

Technological Advancements in Alarm Systems

Modern alarm systems have made significant technological advancements, incorporating "smart" capabilities for precise device identification and location tracking. They have complex data structures to manage and communicate detailed information about each device's status and location, providing enhanced situational awareness and enabling quicker, more effective responses to emergencies. These systems often integrate with other smart technologies, but little standardization has been developed to format the communication and make it universally available. This SAFER proposal offers the first fire alarm reporting solution for the wide range of possible network environments and imposes no requirements that require reduction of a site's security and safety measures.

SAFER Protocol Structure

Explanation of the JSON structure

The SAFER protocol structure leverages JSON's hierarchical format to organize alarm event data methodically. It includes key-value pairs for essential information like customer ID and panel ID, and nested objects for detailed, optional data like GPS coordinates and system categories. This structure ensures comprehensive data representation and offers flexibility to include additional details as needed, making it adaptable and scalable for the diverse needs of modern alarm systems.

Description of Key Components

The SAFER protocol's key components include these four categories:

1. **"CUSTOMER": and "PANEL_ID":** These values index which system is reporting. The values are text strings that can be very specific in SAFER, not limited as in Contact ID's four or two HEX characters.
2. **Device Identifiers. ("DEVICE_ID":, "DEVICE_TYPE":, and "DEVICE_LOCATION":)** Together, these three values uniquely identify each device in a system, with DEVICE_ID indicating the electrical identity, and DEVICE_TYPE specifying the kind of device (e.g., Pendant Transmitter). DEVICE_LOCATION is the installer's descriptive reason for why the device is installed, and historically contains abbreviated text. As above, values are text with no character limits and should be written to match the data as shown on site. Note that DEVICE_LOCATION should match what the field tech programmed into the panel on site. An improved Owner Location should be located in the field named "**OWNER_LOCATION.** "
3. **"STATUS":** Numeric codes representing the device's current status, ensuring immediate and clear communication about the nature of the event or alert. At this introductory phase, we start with the Contact ID standards, and add codes for Supervisory, Trouble, and Status:

   1=Active, 2=Supervisory, 3=Restore, 4=Trouble, 5=Status only, and 6=Previously reported
4. **"SAFER_ID":** for message identification and version control. The "SAFER_ID" section comprises two fields: "version" and "unique_id". The "version" field specifies the version of the SAFER message format being used, facilitating compatibility and future updates. The "unique_id" field provides a globally unique identifier for each message, ensuring that every message can be distinctly tracked and managed, significantly enhancing the traceability and auditability of the system's communication.

Event Details. (Optional) A nested structure that can include various sub-components such as:

- **"OWNER_LOCATION":** An optional detailed, and more humanly-readable description of the location, aiding in swift response and precise navigation in complex environments.
- **"PANEL_GPS":** Exact geographical coordinates of the alarm panel.

- **"CONDITION":** Specifics about the event or device condition prompting the alert.
- **"SYSTEM_CATEGORY":** The system categories like Medical, Fire, Security, etc.
- **"ZONE":** Details about the specific zone of the event, potentially including a descriptive URL for more information. Start by adding the name and relevant descriptions of the zone that might be helpful.
- **"GROUP":** Information about the larger grouping the device belongs to, like a building or campus section. Some systems may not have any logical groupings. If you do, you'll know because you have more information gathered and using groups will give you a way to label them. Start by adding the name and relevant descriptions of the group that might be helpful.
- **"CONTACT_ID":** Allows systems still using the Contact ID standard to receive and understand the basic event information.

Examples of SAFER Alarm Messages

Each component of the SAFER protocol is designed to provide comprehensive, clear, and actionable information, which makes it a robust and versatile framework for modern alarm systems.

Here is an example of a SAFER message with only the essential fields for a smoke alarm event, plus optional fields for Contact ID:

```
{
  "CUSTOMER": "5547785214556987452",
  "PANEL_ID": "Node 01",
  "DEVICE_ID": "22-011",
  "DEVICE_TYPE": "Smoke Detector",
  "DEVICE_LOCATION": "2FL CONF. RM",
  "STATUS": 1,
  "SAFER_ID": {
    "version": "0.0.0",
    "unique_id": "223e4567-e89b-12d3-a456-426614174000"
  },
  "OPTIONAL": {
    "CONTACT_ID": {
      "version": "SIA DC-05-1999.09",
      "code": "1111181111220118"
    }
  }
}
```

The message includes the customer and panel identifiers, the device ID and type, and the status of the alarm. It also incorporates a Contact ID compatibility field, ensuring backward compatibility with systems still using the Contact ID standard.

Here's an expanded version of the same SAFER message with plausible values filled in for the optional fields for a smoke alarm:


```
{
  "CUSTOMER": "5547785214556987452",
  "PANEL_ID": "Node 01",
  "DEVICE_ID": "22-011",
  "DEVICE_TYPE": "Smoke Detector",
  "DEVICE_LOCATION": "2FL CONF. RM",
  "STATUS": 1,
  "SAFER_ID": {
    "version": "0.0.0",
    "unique_id": "323e4567-e89b-12d3-a456-426614174000"
  },
  "OPTIONAL": {
    "PANEL_GPS": {
      "latitude": "37.7749",
      "longitude": "-122.4194",
      "altitude": "15m"
    },
    "LOCATION": "2nd Floor",
    "CONDITION": "Smoke Detected",
    "OWNER_LOCATION": "Conference Room",
    "DATE": "2024-01-31",
    "TIME": "15:45",
    "SYSTEM_CATEGORY": "Fire Alarm",
    "ZONE": {
      "name": "Zone B2",
      "description": "Main office areas",
      "url": "https://examplecompany.com/zoneb2-details"
    },
    "GROUP": {
      "name": "Tech Park Campus",
      "description": "Building A to Building F",
      "url": "https://examplecompany.com/techpark-info"
    },
    "GPS_COORDINATES": {
      "latitude": "37.7749",
      "longitude": "-122.4194",
      "altitude": "15m"
    },
    "CONTACT_ID": {
      "version": "SIA DC-05-1999.09",
      "code": "1111181111220118"
    }
  }
}
```

Here is an example with information that would have been Contact ID code 130, a burglary.


```
{
  "CUSTOMER": "7723458762558899453",
  "PANEL_ID": "Node 02",
  "DEVICE_ID": "45-030",
  "DEVICE_TYPE": "Intrusion Detector",
  "DEVICE_LOCATION": "Main Entrance",
  "STATUS": 1,
  "SAFER_ID": {
    "version": "0.0.0",
    "unique_id": "423e4567-e89b-12d3-a456-426614174000"
  },
  "OPTIONAL": {
    "PANEL_GPS": {
      "latitude": "40.7128",
      "longitude": "-74.0060",
      "altitude": "10m"
    },
    "LOCATION": "Front Lobby",
    "CONDITION": "Unauthorized Entry",
    "OWNER_LOCATION": "Main Building - Ground Floor",
    "DATE": "2024-01-31",
    "TIME": "22:30",
    "SYSTEM_CATEGORY": "Burglary",
    "ZONE": {
      "name": "Zone A1",
      "description": "Entry and front desk area",
      "url": "https://examplesecurity.com/zonea1-details"
    },
    "GROUP": {
      "name": "Downtown Facility",
      "description": "All entry and exit points",
      "url": "https://examplesecurity.com/downtown-facility"
    },
    "GPS_COORDINATES": {
      "latitude": "40.7128",
      "longitude": "-74.0060",
      "altitude": "10m"
    },
    "CONTACT_ID": {
      "version": "SIA DC-05-1999.09",
      "code": "1301181301450308"
    }
  }
}
```

Sample Formatted Template

This template is a structured form where software can dynamically replace placeholders with actual data values from the database. This automated fill-in process allows for the efficient generation of detailed, accurate messages for different alarm states, streamlining communication and ensuring precise monitoring within the system. The first part holds instructions in a comment and is followed by examples for fire alarm, supervisory, and trouble formats.

```
<?xml version="1.0" encoding="utf-8"?>

<outputFormatConfiguration>
<!--
This file Must be named FormatedOutput.XML and must be located in the same folder as
RescueLogic’s System Monitor. Restart System Monitor after making changes to this
file. All values in this file are case-sensitive.

Formats for device information fields to be obtained from the database are:
             ![d.PanelDescription]
             ![d.Address] = database.Address
             ![d.Description] = database.Description
             ![d.PanelID]
             ![d.DeviceType]
             ![d.ConditionText]
             ![d.GroupName]
             ![d.ZoneName]
             ![d.Layout]

A formatted panel reset message will be generated from the resetFormat key.
             Fields used for global reset messages are:
             ![PanelDescription]
             ![PanelID]

       To Include Time and date, use one of the following:
             ![SystemDateTime="mm/dd/yyyy hh:mm:ss"]
             ![DeviceDateTime="mm/dd/yyyy hh:mm:ss"]

             Format syntax:            Sample:
             m/d/yy              1/27/93
             dddd, mmmm dd, yyyy Wednesday, January 27, 1993
             d-mmm               27-Jan
             mmmm-yy             January-93
             hh:mm AM/PM         07:18 AM
             h:mm:ss a/p         7:18:00 a
             d-mmmm h:mm         3-January 7:18
```

```
Each of these keys identify the string format for Logical States found in the
database:
             logicalStateNormal
             logicalStateFireAlarm
             logicalStateSupervisory
             logicalStateTrouble

       Each of these keys will generate default messages when the Device Type is not
yet selected.
             baseStateNormal
             baseStateAlarm
             baseStateTrouble
-->

<logicalStateFireAlarm>
{
  "CUSTOMER": "5547785214556987452",
  "PANEL_ID": "![d.PanelID]",
  "DEVICE_ID": "![d.Address]",
  "DEVICE_TYPE": "![d.DeviceType]",
  "DEVICE_LOCATION": "![d.Description]",
  "STATUS": 1,
}
</logicalStateFireAlarm>


<logicalStateTrouble>
{
  "CUSTOMER": "5547785214556987452",
  "PANEL_ID": "![d.PanelID]",
  "DEVICE_ID": "![d.Address]",
  "DEVICE_TYPE": "![d.DeviceType]",
  "DEVICE_LOCATION": "![d.Description]",
  "STATUS": 4,
}
</logicalStateTrouble>

<logicalStateSupervisory>
 {
 "CUSTOMER": "5547785214556987452",
  "PANEL_ID": "![d.PanelID]",
  "DEVICE_ID": "![d.Address]",
  "DEVICE_TYPE": "![d.DeviceType]",
  "DEVICE_LOCATION": "![d.Description]",
  "STATUS": 2,
}
</logicalStateSupervisory>

</outputFormatConfiguration>
```

End-User Interaction

This section describes why end users will find the SAFER standard systems intuitive, reliable, and responsive to their needs. It describes how the standard considers the practical, day-to-day experience of those who interact with fire alarm systems, whether they're monitoring, maintaining, or responding to them.

A standard that's not just technically sound but also user-friendly recognizes the importance of clear communication, ease of use, and meaningful alerts. This section outlines several principles and practices that make the SAFER standard accessible and understandable to all users, regardless of their technical background.

SAFER Reports

End-users will interact with SAFER primarily through user-friendly interfaces, designed to display the comprehensive data in an accessible and actionable manner. Alerts, device statuses, and detailed event information will be clearly presented, allowing for quick understanding and response to incidents. Customizable views and filters will enable users to focus on the most relevant data for their needs.

Customization Options

For customization, SAFER offers optional fields to cater to various user preferences and requirements. This may include customizable alert thresholds, selective data display options, and user-defined notification settings. Users will be able to tailor the dashboard layout, choose specific data points to monitor, and set up personalized alerts to ensure they receive relevant and actionable information tailored to their unique operational needs.

Troubleshooting

These are common issues to look out for in SAFER:

- **Data mismatch.** Verify the accuracy of database entries against SAFER reports to resolve inconsistencies.
- **Connectivity problems.** Check network connections and configurations if SAFER reports are not being received as expected.
- **Incorrect alerts.** Review system settings and thresholds for alert triggers to ensure they align with actual conditions.
- **User interface issues.** Refresh or update the interface if display errors occur, and ensure compatibility with your system's software version.
- **Report delays.** Investigate any lags in report generation or delivery, starting with system performance checks and network speed tests.

Implementation Guidelines

This section is intended to ensure that the transition to and adoption of this innovative standard is both smooth and impactful.

Every stakeholder, from manufacturers to installers to end-users, should be able to understand the "what" and the "how" of this transformation. This section is designed to demystify the process, providing a clear, structured, and comprehensive roadmap for implementation.

Implementing a new standard is a multifaceted endeavor and requires an understanding of both the theoretical framework and the practical, and on-site realities of fire alarm systems. These implementation guidelines are crafted with the recognition that effective implementation is the linchpin in realizing the potential of the SAFER standard.

This section outlines a set of instructions to bridge the conceptual and the practical. It acknowledges the diversity of contexts in which the SAFER standard will be applied and aims to provide flexible yet robust guidance that can be adapted to various scenarios.

Guidance for Manufacturers

In the testing phase, use tools like HyperTerminal or PuTTY to connect to the configured TCP/IP port. These applications can help simulate the communication environment and allow you to visually verify the SAFER message's structure and data integrity. Connect to the port, send a SAFER message, and observe the output on the terminal to ensure correct data formatting and transmission.

For manufacturers implementing the SAFER protocol, follow these steps:

1. **Update system software.** Ensure your alarm system's software supports JSON formatting and TCP/IP communication.
2. **Define data structure.** Integrate the SAFER data structure into your system, ensuring all required and optional fields are correctly formatted.
3. **Develop data generation logic.** Implement logic in your system to populate SAFER messages with real-time data from alarm events.
4. **Set up TCP/IP communication.** One way to test the output of your product is to Configure your system to send SAFER messages to a TCP/IP port defined for a Windows Printer with the configuration: Manufacturer: Generic, Model: Text-Only.
5. **Testing.** Test the output by sending SAFER messages to the configured port and verify correct message formatting and accurate data representation.
6. **Iterate and improve.** Based on testing results, make necessary adjustments to ensure reliable and accurate message generation and transmission.

Tips for Integrating SAFER into Existing Systems

When integrating SAFER into existing systems:

1. **Compatibility check.** Ensure your system's hardware and software can support JSON formatting and TCP/IP communication.
2. **Data mapping.** Identify how existing data points correspond to SAFER fields, paying attention to maintaining data integrity and accuracy.
3. **Gradual integration.** Start with one system component, test thoroughly, then scale up, ensuring each step is fully functional before moving on.
4. **Training.** Provide comprehensive training for your team on the SAFER format and the updated system's functionalities.
5. **Monitor and iterate.** Continuously monitor the performance post-integration and be ready to make adjustments based on feedback and evolving requirements.

Best Practices for Data Integrity and Security

For data integrity and security, especially in shared networks and life safety network management consider these factors:

1. **Encryption.** Utilize strong encryption for data in transit and at rest to protect sensitive information from unauthorized access.
2. **Access control.** Implement strict access control policies, ensuring only authorized personnel can access or modify the alarm system data.
3. **Regular audits.** Conduct regular security audits and vulnerability assessments to identify and mitigate potential risks.
4. **Network segmentation.** Separate life safety networks from general-purpose networks to reduce the risk of cross-contamination and ensure dedicated bandwidth.
5. **Data validation.** Implement rigorous data validation checks to prevent incorrect or malicious data from compromising the system.
6. **Incident response plan.** Have a robust incident response plan in place to quickly address any security breaches or data integrity issues.

Note: Also see NFPA 72, 23.6.3.5 Management Organization

Software Development Integration

This section offers resources familiar to every seasoned developer for integration of the SAFER standard into various systems. Tailored for the developer's toolkit, we chose widely used techniques like parsing JSON structures and harnessing regular expressions. This section is a nod to common development practices, providing clear guidelines and examples that align with everyday coding scenarios. The aim is to leverage standard skills to foster a seamless and robust implementation of the SAFER standard.

Guidelines for Parsing and Generating SAFER messages

For developers integrating SAFER:

- **Understand JSON structure.** Be familiar with JSON syntax and the specific structure of SAFER messages.
- **Parsing libraries.** Utilize robust libraries or frameworks for parsing JSON in your chosen development language to ensure reliable data extraction.
- **Generating SAFER messages.** Implement logic to generate SAFER messages from system data, ensuring all fields are populated accurately.
- **Validation.** Include schema validation to ensure the generated SAFER messages adhere to the defined structure and data types.
- **Testing.** Rigorously test parsing and generation of SAFER messages under various scenarios to ensure reliability and accuracy.

Output Example: Set a Template and Replace

You can send SAFER messages formatted using this technique. To create a flexible and dynamic document structure, set a template of text with the desired JSON format and then "Replace" the placeholder variables with real-time values. Developers can use the Replace function in their code to substitute these placeholders with actual database values. This approach ensures the document can be dynamically generated based on current data, maintain a consistent format, and provide the versatility needed in various application contexts.

See the Appendix for an example FormattedOutput file template that RescueLogic® uses to send alarm data to pagers, BMS systems, and Mass Notification systems.

Sample Code

The common “replace” function is familiar to programmers populates the template with data. If you’re not a programmer, this section is not required reading.

Where ‘s’ is the template string and db.Address is the database value for the device ID, this is example code to replace the variable in the template string.

```
s = FormattedOutput_logicalStateFireAlarm
Replace(s, "![d.Address]", (db.Address))
```

The syntax `Replace(s, "![d.Address]", (db.Address))` is used in languages like Visual Basic (VB) and other Basic-like languages. It’s a straightforward function call for string replacement, commonly seen in languages that prioritize readability and straightforward syntax for string operations.

In Java, you would typically use the `replace` method to replace parts of a string. Here’s the given code in Java:

```
String s = FormattedOutput_logicalStateFireAlarm;
s = s.replace("![d.Address]", db.getDeviceID());
```

Similarly, `s` is assigned the original template string, and `db.getDeviceID()` would be a method call that returns the Device_ID string from object `db` and its function named ‘getDeviceID’.

Input Example: Regular Expressions

You can read SAFER messages formatted using this technique. Most computer languages can use Regular Expressions. Here’s how to extract the "CUSTOMER" value from a filled-in SAFER form. You would typically use a regular expression that searches for the "CUSTOMER" key and captures the associated value. Below is a brief instruction followed by a code example:

Instructions

1. Identify the Pattern: Locate the "CUSTOMER" key in the JSON-like structure of the SAFER form.

2. Create the Regular Expression: Use a regular expression that matches the "CUSTOMER" key and captures the value that follows.

3. Apply the Regular Expression: Use the regular expression on the SAFER form text to find matches.

4. Extract the Value: Extract the captured group, which contains the customer value.

Sample Code

Assuming the SAFER form is in a JSON-like format and the "CUSTOMER" value is a string:

```python
import re

# Sample text from a SAFER form
safer_form_text = ‘‘‘

{
    "CUSTOMER": "7747785214556987452",
    "PANEL_ID": "Fire Alarm Node 1",
    // other key-value pairs...
}
‘‘‘

# Regular expression to match the "CUSTOMER" key and capture the associated value
pattern = r’"CUSTOMER":\s*"([^"]+)" ‘

# Find matches using the regular expression
matches = re.search(pattern, safer_form_text)

# Extract the customer value
if matches:
    customer_value = matches.group(1)
    print(f’Customer: {customer_value}’)
else:
    print(‘Customer key not found.’)
```

In this code:

- The regular expression `r’"CUSTOMER":\s*"([^"]+)"‘` is looking for the pattern `"CUSTOMER":` followed by zero or more spaces (`\s*`) and then a double-quoted string. The parentheses `()` capture the contents of the double-quoted string.
- `re.search` searches the `safer_form_text` for the pattern.
- `matches.group(1)` extracts the value captured by the parentheses in the regular expression.

Remember to test this with the actual structure of your SAFER forms as JSON structures may vary, and adjust the regular expression accordingly.

Compliance and Testing Requirements

Compliance and testing systems are the bedrock of trust and reliability. This section should be integral to the standard, serving to guide manufacturers, installers, and end-users through the essential processes that validate the SAFER standard's efficacy and integrity.

Compliance aligns with industry regulations and best practices. Testing evaluates every component, ensuring that they perform in real-world scenarios. This section outlines the protocols, methodologies, and tools necessary to meet the stringent expectations set forth by the industry.

A Technical Committee governing this document could align these requirements with standard practices.

Through a concise yet comprehensive approach, the "Compliance and Testing Requirements" section empowers stakeholders to navigate the complexities of regulatory adherence and quality assurance with confidence and clarity. Here, we lay the groundwork for a system that doesn't just function but excels, reinforcing the SAFER standard's commitment to excellence, safety, and reliability.

For SAFER compliance and testing:

1. **Standard adherence:** Ensure all SAFER messages strictly adhere to the defined JSON structure.
2. **Data accuracy:** Validate that data fields are accurately populated, reflecting the true status of devices and alarms.
3. **Security protocols:** Implement and maintain robust security measures, including data encryption and secure communication channels.
4. **Interoperability testing:** Regularly test SAFER messages with different systems to ensure compatibility and seamless data exchange.
5. **Documentation and training:** Maintain comprehensive documentation and provide training to ensure all stakeholders understand the compliance requirements and procedures.

Testing Methodologies and Tools

For testing methodologies and tools, consider:

- **Automated testing suites.** Use automated tools to simulate various scenarios and validate the structure and data integrity of SAFER messages.
- **Unit testing.** Implement unit tests for individual components to ensure each part functions correctly independently.
- **Integration testing.** Test the interaction between different system components with SAFER messages to ensure smooth data flow and handling.
- **Stress testing.** Simulate high load conditions to ensure the system maintains performance and reliability.

- **User acceptance testing.** Involve end-users in testing to ensure the system meets their requirements and expectations.

Certification and Compliance Considerations

The SAFER protocol defines an event data structure only. It does not constitute a listed product, a supervisory pathway classification, or a code-compliant system configuration by itself.

Compliance with NFPA standards, UL/ULC listings, local authority requirements, and other regulatory frameworks remains the responsibility of system designers, manufacturers, and installers.

Implementation of SAFER within a product or system may require evaluation under applicable product safety standards, including but not limited to UL 864 or related standards, depending on the application.

The publication of this protocol does not imply certification, approval, or endorsement by any standards development organization or regulatory authority.

Future Prospects and Upgrades

The SAFER protocol is intended to evolve through open publication and industry participation. Because the event structure is defined using extensible JSON conventions, additional fields and clarifications may be incorporated without disrupting existing implementations.

Future revisions may address:

- Refinement of field definitions based on implementation experience
- Optional schemas for specialized use cases
- Version management practices
- Security guidance enhancements
- Interoperability testing feedback

Governance of future revisions may occur through an open working group, standards development organization, or collaborative industry forum. No single entity controls adoption. Implementers are encouraged to reference version identifiers to ensure compatibility.

The objective of SAFER is stability of core structure combined with flexibility for future adaptation.

As networking, analytics, and device ecosystems evolve, the protocol may expand to accommodate additional device categories, metadata structures, or data validation mechanisms. Backward compatibility principles should guide any substantive changes.

JSON-based message structures are inherently extensible. This characteristic allows new fields to be added without invalidating existing implementations, provided versioning and documentation practices are maintained.

Roadmap for Future Enhancements

The nature of SAFER allows for continuous enhancements in data representation, security features, and integration capabilities. Future enhancements may include additional schema definitions, security guidance, expanded device classifications, or formal interoperability test profiles. Presumably, new fields will be created for expansion and relatable to a broadening range of devices and systems. The focus can evolve with user-centric upgrades, ensuring that the standard evolves to meet the dynamic needs of the industry and the end users.

Adapting to Evolving Technology

The SAFER protocol will adapt to evolving technology because it has flexible data structures. It allows seamless integration of new device types and data sources. It prioritizes interoperability and

scalability, ensuring compatibility with new industry standards and technologies. Regular use and community-driven enhancements will ensure SAFER remains at the forefront of technological advancements, offering a robust and future-proof solution for alarm system reporting and management.

Programmers and IT professionals say a JSON form like SAFER is extensible by nature. That means it is inherently adaptable. The flexible design allows for the easy addition of new data fields as technologies evolve, without disrupting existing systems. Its design supports a modular approach, so you can extend or modify data schemas to accommodate emerging devices and standards. This means that as integration needs grow and change, SAFER can be swiftly adapted to meet those demands, and be used in evolving system environments.

Glossary of Terms

These terms encompass key concepts from the SAFER structure, implementation, and integration processes, as well as general IT and networking terminologies relevant to understanding and applying the SAFER design:

API (Application Programming Interface) is a set of protocols, routines, and tools for building software and applications. It specifies how software components should interact, allowing different systems to communicate with each other. APIs are crucial for enabling the integration of new features and services, making them essential for modern software development and system interoperability.

Compliance Verification is the process of ensuring that a product, service, system, or organization meets the required standards, regulations, or specifications. It involves a thorough review and testing of components and procedures to ensure they align with established guidelines, often conducted by external agencies or internal departments responsible for upholding standards and maintaining quality and safety in operations.

Data Validation is the process of verifying the accuracy and quality of data before it is used in a system or database. It involves checking the data against predefined rules and standards to ensure it is correct, meaningful, and secure. This process is crucial to prevent data errors, maintain data integrity, and ensure reliable data processing and analysis in any system.

Device Identifiers (non-required) in SAFER, such as PANEL_GPS and CONDITION, are optional labels or data points that provide added context-specific information about a device or event. While not mandatory for the basic functioning of the system, they offer enhanced detail and situational awareness, enriching the data set for more comprehensive monitoring and analysis. These identifiers allow for a more nuanced understanding of the device's environment or status.

Device Identifiers (required) in SAFER are the unique codes assigned to each individual device within an alarm or security system and are mandatory. These identifiers are critical for the accurate reporting, monitoring, and management of devices, ensuring that each device can be distinctly identified, and its data precisely logged and analyzed in the system. "CUSTOMER" "PANEL_ID" "DEVICE_ID" "DEVICE_TYPE" "DEVICE_LOCATION" "STATUS"

Encryption is a security process where information is encoded so that only authorized parties can access it. It uses algorithms to transform readable data (plaintext) into an unreadable format (ciphertext), protecting it from unauthorized access, disclosure, alteration, or destruction. Encryption is vital for securing sensitive data, ensuring privacy, and maintaining data integrity in various digital communications and transactions.

Extensible refers to SAFER's inherent adaptability and capacity to be extended or expanded. It denotes the ability of the format to incorporate additional data fields and adapt to emerging technologies. It also means the design will integrate new device types or data sources without disrupting the existing infrastructure. This flexibility ensures that SAFER can evolve and scale in response to the changing needs and advancements in the alarm and security industry.

GPS (Global Positioning System) is the satellite-based navigation system providing geolocation and time information to GPS receivers anywhere on or near the Earth. It operates independently of

any telephonic or internet reception and provides critical capabilities to military, civil, and commercial users globally.

Incident Response Plans are a structured approach for handling and managing the aftermath of a security breach or cyberattack. The plan includes preparedness measures, detailed steps for detection, analysis, containment, eradication, and recovery, as well as post-incident procedures. It aims to minimize damage, recover operations swiftly, and mitigate risks. The plan involves clear roles and responsibilities, communication protocols, and documentation processes to ensure a coordinated and effective response to security incidents.

Interoperability Testing is the process of evaluating the ability of different systems, components, or services to work together effectively. It focuses on verifying that interfaces between systems are correctly implemented and communicating as intended, ensuring that different parts of a system, possibly from different vendors, can cooperate seamlessly in a given environment, ultimately ensuring compatibility and functional consistency across diverse system components.

IP Addresses (Internet Protocol Address) are a unique numerical label assigned to each device connected to a computer network that uses the Internet Protocol for communication. It serves two primary purposes: identifying the host or network interface and providing the location of the host in the network, thereby facilitating the routing of data and information from one device to another.

IP Ports are a numerical identifier in network communications that, when combined with an IP address, uniquely identifies a specific process or service on a device. It enables the device to support multiple, simultaneous communication sessions by allowing data traffic to be directed to the appropriate application or service.

IP Sockets are a network communication endpoint that allows devices or programs to exchange data and connect over an IP network. It provides a mechanism for establishing, managing, and terminating connections, enabling bidirectional communication streams between servers and clients, based on the specific IP address and port number. Sockets are crucial in network programming for ensuring structured and reliable data transmission.

JSON (JavaScript Object Notation) is a lightweight, text-based data format that is easy for humans to read and write, and easy for machines to parse and generate. It's based on a subset of the JavaScript language but is language-independent, making it widely used in programming for data interchange. JSON structures data in a clear, hierarchical format using name/value pairs and ordered lists, facilitating organized and efficient data communication.

Network Segmentation is a security strategy that involves dividing a computer network into smaller parts, or segments, to isolate groups of devices and manage traffic more efficiently. This practice enhances security by limiting the spread of potential breaches to just one segment, reduces congestion by controlling the flow of network traffic, and simplifies troubleshooting by isolating issues within specific segments.

SAFER (Security And Fire Emergency Report) is a structured, JSON-based format designed for detailed and precise communication of alarm events in modern fire alarm and security systems. It enhances data representation, interoperability, and system integration, ensuring a robust and

adaptable framework for conveying comprehensive alarm information, including device specifics and statuses, in a clear and actionable manner.

SDK (Software Development Kit) is a collection of software tools, libraries, relevant documentation, code samples, processes, or guides that developers use to create applications on specific platforms. SDKs streamline the development process by providing essential building blocks, allowing developers to focus on creating unique functionalities rather than building everything from scratch.

Status Codes in SAFER, akin to Contact ID Status Codes, serve as essential numeric representations of specific states or conditions of devices within a security system. They offer a succinct, standardized way to communicate diverse statuses, ensuring clarity and consistency in monitoring and response, while also accommodating a broader range of states to reflect the complexities of modern alarm systems. 1=Active, 2=Supervisory, 3=Restore, 4=Trouble, 5=Status only, and 6=Previously reported

TCP/IP (Transmission Control Protocol/Internet Protocol) is the foundational communication protocol suite for the Internet and similar networks. TCP ensures reliable, ordered, and error-checked delivery of data between applications, while IP handles the routing and addressing of packets to ensure they reach their intended destination. Together, TCP/IP enables interconnected devices to communicate and exchange data over networked systems.

Reference Materials and Resources

Contact ID Protocol for Alarm System Communications

This document outlines the Contact ID protocol, a standard for communication between alarm systems and monitoring centers. It details the structure, codes, and transmission sequence used in alarm reporting. *Source: securityindustry.org*

Class N - Networks for Fire Alarm and Mass Notification Systems

This booklet provides comprehensive guidelines and standards for NFPA to use Ethernet in fire alarm systems. It includes detailed information on system requirements, installation procedures, and maintenance protocols. *Source: rescuelogic.com/publications*

NFPA 72: National Fire Alarm and Signaling Code

NFPA 72 provides the latest safety provisions to meet society's changing fire detection, signaling, and emergency communications demands. It covers the application, installation, location, performance, and maintenance of fire alarm systems and their components. *Source: nfpa.org*

NFPA 950, Standard for Data Development and Exchange for the Fire Service

NFPA 950 is designed to standardize data for operational information sharing in the Fire Service, and it also gives deference to the JSON format proposed to be used by SAFER. *Source: nfpa.org*

RescueLogic® Users Guide

The *RescueLogic® Users Guide* offers comprehensive instructions on utilizing the RescueLogic® software for alarm system management. It includes detailed steps for setup, operation, and troubleshooting, ensuring users can fully leverage the software's features for effective monitoring and management of alarm systems. *Source: rescuelogic.com/publications*

Contact Information

Dan Horon: dan@rescuelogic.com

Jeff Horon: jeff@rescuelogic.com

Contact and Stewardship Information

Questions, comments, and implementation feedback regarding this specification may be directed to:

Dan Horon
dan@rescuelogic.com

Jeff Horon
jeff@rescuelogic.com

The SAFER Protocol is published as an open specification under the Apache License 2.0. Future revisions may incorporate implementation feedback, technical clarifications, and industry collaboration.

Publication of this document does not imply endorsement by NFPA, UL, or any other standards development organization.

Implementers are encouraged to reference the version identifier included in SAFER messages to ensure compatibility with the applicable specification revision.